DATABASE MANAGEMENT SYSTEMS · PROJECT · PHASE 1

MUSE — Music Mood Intelligence & Recommendation Database System

ER Diagram and Normalisation Report

Submitted by	
Roll number	
Programme	B.Tech CSE
Course	Database Management Systems
Submitted to	
Date	

DBMS: MySQL 8.0 (InnoDB). All entities, attributes, keys and relationships in this report are taken from the project's executable schema (baseline + migrations 0001–0003).

Contents

1. Project topic
2. Entities
3. Relationships and cardinalities
4. ER diagram (16 core entities) — crow's foot, Chen and textbook notation
5. Normalisation: UNF → 1NF → 2NF → 3NF
6. Final normalised schema
7. Summary
8. Appendix A — Attributes of each entity
9. Appendix B — The remaining 48 tables of the database
10. Appendix C — Complete ER diagram of all tables

1. Project topic

MUSE is a database system for a music streaming service that recommends songs by **mood**. It stores a music catalogue (songs, artists, albums, genres), analyses each song's audio profile to tag it with moods, records what each user listens to, learns each user's taste from that history, and generates ranked, explained recommendation lists. Users can also build playlists.

Problem addressed. A plain song list cannot answer questions such as "suggest calm songs I have not heard", "why was this song recommended?" or "what is this user's favourite mood this week?". These need well-structured data about songs, moods, listening behaviour and recommendations that stays consistent as it grows.

Objectives of the database.

- Store the catalogue once, without duplication (each song, artist and album is one row).
- Link songs to many genres and many moods, with a confidence score for each mood.
- Let users rate how well each song fits each mood on a 0–100 % scale, kept apart from the AI's own mood prediction.
- Record every play inside a listening session, so taste can be learned from real behaviour.
- Store each recommendation list with the score of every song, so that recommendations can be explained later.
- Enforce integrity in the database itself: primary, unique and foreign keys, CHECK constraints and triggers.

Scope of this report. The full implementation has 64 tables and 124 foreign keys. For clarity this report models the **16 core entities** that carry the main business data, with **all** their 162 attributes and the 24 relationships between them. Every other table supports one of these 16 (for example, `roles` supports `users`); Appendix B lists them, and Appendix C shows the complete diagram.

2. Entities

The entities fall into six functional areas. An **associative entity** is one that exists to resolve a many-to-many relationship and can carry attributes of its own (for example the confidence of a song's mood).

Entity (table)	Meaning	Primary key	Other candidate keys (UNIQUE)	Attrs
Users				
users User	A login account of a listener, curator or admin.	user_id	(email); (username)	11
user_profiles User profile	Display and personal settings of one user (split 1:1 from the account).	user_id	—	7
Music Catalogue				
artists Artist	A singer, band or composer.	artist_id	(normalized_name, disambiguation)	11
albums Album	An album, EP, single or film soundtrack.	album_id	—	9
songs Song	One track in the catalogue.	song_id	(primary_artist_id, normalized_title, version_label); (isrc)	18
genres Genre	A music genre; genres form a hierarchy (sub-genre of).	genre_id	(name)	5
song_genres Song–Genre (associative)	Resolves the M:N between songs and genres.	song_id, genre_id	(primary_song_key)	4
Mood Intelligence				

moods Mood	A mood such as Calm or Energetic, with its target audio profile.	mood_id	(name)	13
song_moods Song–Mood (associative)	Resolves the M:N between songs and moods; one row per song, mood and analysing model.	song_id, mood_id, model_id	—	6
song_mood_ratings Mood-match rating (associative)	One user's 0–100 % judgement of how well one song fits one mood (ternary User–Song–Mood). Separate from the AI's song_moods.	user_id, song_id, mood_id	—	7
Listening History				
listening_sessions Listening session	One continuous period of listening by a user.	session_id	(session_id, user_id)	10
listening_history Play (listening history)	One play of one song inside a session.	play_id	—	13
Recommendations				
recommendation_batches Recommendation batch	One request for recommendations and the targets it used.	batch_id	—	20
recommendation_items Recommended song (associative)	One ranked song in a batch, with all its scores; resolves batch M:N song.	batch_id, song_id	(batch_id, rank_position)	13
Playlists				
playlists Playlist	A user-owned playlist.	playlist_id	(user_id, name)	11
playlist_songs Playlist–Song (associative)	Resolves the M:N between playlists and songs.	playlist_id, song_id	—	4

Kinds of attributes used: **key** attributes (PK/UK), **simple** attributes (e.g. `title`), **derived** attributes whose value is computed from other data (e.g. `completion_pct`, marked [d] in Section 6), and **multi-valued** facts (a song's genres and moods), which are not stored as columns but as rows of associative entities. The full attribute list with data types is in Appendix A.

3. Relationships and cardinalities

Each relationship below is implemented by a real foreign key. **Cardinality** is written parent : child. **Participation** refers to the child (many) side: *total* means every child row must be linked (the foreign key is NOT NULL); *partial* means the link is optional (nullable foreign key). On the parent side participation is always partial: for example, an artist may exist before any of its songs are added.

#	Relationship	Parent (one)	Child (many)	Card.	Meaning	Child partic.	Foreign key
1	**has profile (1:1)**	User	User profile	1 : 1	Each user profile → exactly one user; each user → at most one user profile	total	`user_profiles(user_id)` → `users`
2	**rates mood fit**	User	Mood-match rating (associative)	1 : N	Each mood-match rating → exactly one user; each user → zero or many mood-match ratings	total	`song_mood_ratings(user_id)` → `users`
3	**opens**	User	Listening session	1 : N	Each session → exactly one user; each user → zero or many sessions	total	`listening_sessions(user_id)` → `users`
4	**receives**	User	Recommendation batch	1 : N	Each batch → exactly one user; each user → zero or many batches	total	`recommendation_batches(user_id)` → `users`
5	**owns**	User	Playlist	1 : N	Each playlist → exactly one user; each user → zero or many playlists	total	`playlists(user_id)` → `users`
6	**performs**	Artist	Song	1 : N	Each song → exactly one artist; each artist → zero or many songs	total	`songs(primary_artist_id)` → `artists`
7	**contains**	Album	Song	1 : N	Each song → zero or one album; each album → zero or many songs	partial	`songs(album_id)` → `albums`
8	**merged into**	Song	Song	1 : N (recursive)	Each song → zero or one song; each song → zero or many songs	partial	`songs(merged_into_song_id)` → `songs`
9	**classified as**	Song	Song–Genre (associative)	1 : N	Each song-genre row → exactly one song; each song → zero or many song-genre rows	total	`song_genres(song_id)` → `songs`
10	**tagged with**	Song	Song–Mood (associative)	1 : N	Each song-mood row → exactly one song; each song → zero or many song-mood rows	total	`song_moods(song_id)` → `songs`
11	**mood-rated as**	Song	Mood-match rating (associative)	1 : N	Each mood-match rating → exactly one song; each song → zero or many mood-match ratings	total	`song_mood_ratings(song_id)` → `songs`
12	**played**	Song	Play (listening history)	1 : N	Each play → exactly one song; each song → zero or many plays	total	`listening_history(song_id)` → `songs`
13	**seeded by**	Song	Recommendation batch	1 : N	Each batch → zero or one song; each song → zero or many batches	partial	`recommendation_batches(seed_song_id)` → `songs`
14	**recommends**	Song	Recommended song (associative)	1 : N	Each recommended song → exactly one song; each song → zero or many recommended songs	total	`recommendation_items(song_id)` → `songs`
15	**listed in**	Song	Playlist–Song (associative)	1 : N	Each playlist entry → exactly one song; each song → zero or many playlist entries	total	`playlist_songs(song_id)` → `songs`
16	**sub-genre of**	Genre	Genre	1 : N (recursive)	Each genre → zero or one genre; each genre → zero or many genres	partial	`genres(parent_genre_id)` → `genres`
17	**genre of**	Genre	Song–Genre (associative)	1 : N	Each song-genre row → exactly one genre; each genre → zero or many song-genre rows	total	`song_genres(genre_id)` → `genres`
18	**mood of**	Mood	Song–Mood (associative)	1 : N	Each song-mood row → exactly one mood; each mood → zero or many song-mood rows	total	`song_moods(mood_id)` → `moods`
19	**rated mood**	Mood	Mood-match rating (associative)	1 : N	Each mood-match rating → exactly one mood; each mood → zero or many mood-match ratings	total	`song_mood_ratings(mood_id)` → `moods`
20	**contains plays**	Listening session	Play (listening history)	1 : N	Each play → exactly one session; each session → zero or many plays	total	`listening_history(session_id, user_id)` → `listening_sessions`
21	**came from list**	Recommendation batch	Play (listening history)	1 : N	Each play → zero or one batch; each batch → zero or many plays	partial	`listening_history(batch_id)` → `recommendation_batches`
22	**ranks**	Recommendation batch	Recommended song (associative)	1 : N	Each recommended song → exactly one batch; each batch → zero or many recommended songs	total	`recommendation_items(batch_id)` → `recommendation_batches`
23	**saved from list**	Recommendation batch	Playlist	1 : N	Each playlist → zero or one batch; each batch → zero or many playlists	partial	`playlists(source_batch_id)` → `recommendation_batches`
24	**includes**	Playlist	Playlist–Song (associative)	1 : N	Each playlist entry → exactly one playlist; each playlist → zero or many playlist entries	total	`playlist_songs(playlist_id)` → `playlists`

Many-to-many relationships and how they are resolved

Conceptual M : N	Associative entity	Attributes of the relationship itself
Song M : N Genre	song_genres	is_primary — at most one primary genre per song (unique key primary_song_key)
Song M : N Mood (per analysis model)	song_moods	confidence, feature_id, analyzed_at
Recommendation batch M : N Song	recommendation_items	rank_position and the nine scores (mood, audio, preference, recency, trend, context, regional, exploration, repetition penalty) and final_score
Playlist M : N Song	playlist_songs	position, added_at
User M : N Song M : N Mood (ternary: "rates how well")	song_mood_ratings	match_percentage (0–100, two decimals), review_text, created_at, updated_at — one row per user, song and mood
User M : N Song (listens)	listening_history (an entity with its own key play_id)	started_at, listened_sec, origin, … — a user can play the same song many times, so each play is its own row

> **Special cases.** *One-to-one:* user_profiles.user_id is both its primary key and a foreign key to users, so a user has at most one profile. *Recursive (unary):* genres.parent_genre_id → genres (genre hierarchy) and songs.merged_into_song_id → songs (a duplicate song points to the song it was merged into). *Composite foreign key:* listening_history(session_id, user_id) → listening_sessions(session_id, user_id), so a play can only belong to a session of the same user.

4. ER diagram (16 core entities)

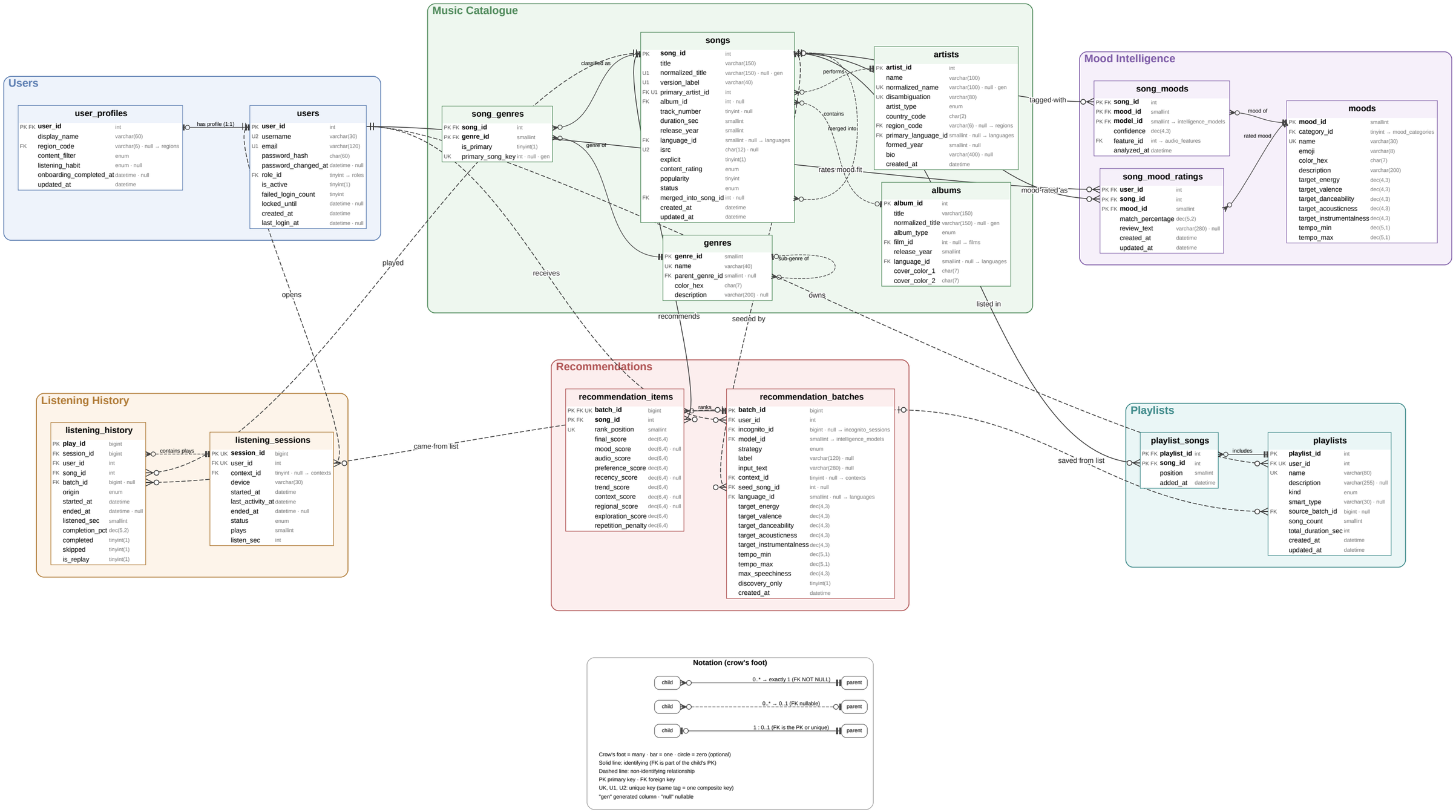
The next page shows the ER diagram (A3 landscape, vector — zoom in for detail). It uses **crow's-foot notation**:

Symbol at the end of a line	Meaning
Two bars ‖	exactly one (mandatory)
Bar and circle |○	zero or one (optional)
Crow's foot with circle ○<	zero or many
Solid line	identifying relationship: the foreign key is part of the child's primary key (associative entities)
Dashed line	non-identifying relationship

Inside each entity box every attribute is listed with its data type. **PK** marks the primary key, **FK** a foreign key, **UK/U1/U2** a unique (candidate) key, *null* an optional attribute and *gen* a generated (derived) column. A foreign key written "→ table" points to one of the supporting tables of Appendix B. All 24 relationships of Section 3 are drawn; the diagram was generated from the live schema and checked automatically (every entity, every column and one line per foreign key).

MUSE — ER Diagram (16 core entities)

16 of the 64 tables, with ALL their columns and the 24 real foreign keys between them. "→ table" marks a foreign key to a table not drawn here.
Simplified view for submission: the database is unchanged; each of the other 48 tables belongs to one of these 16 (see MUSE_ERD_Core_GUIDE.md). Complete schema: MUSE_ERD_Full_Schema.



4.1 The same model in Chen notation

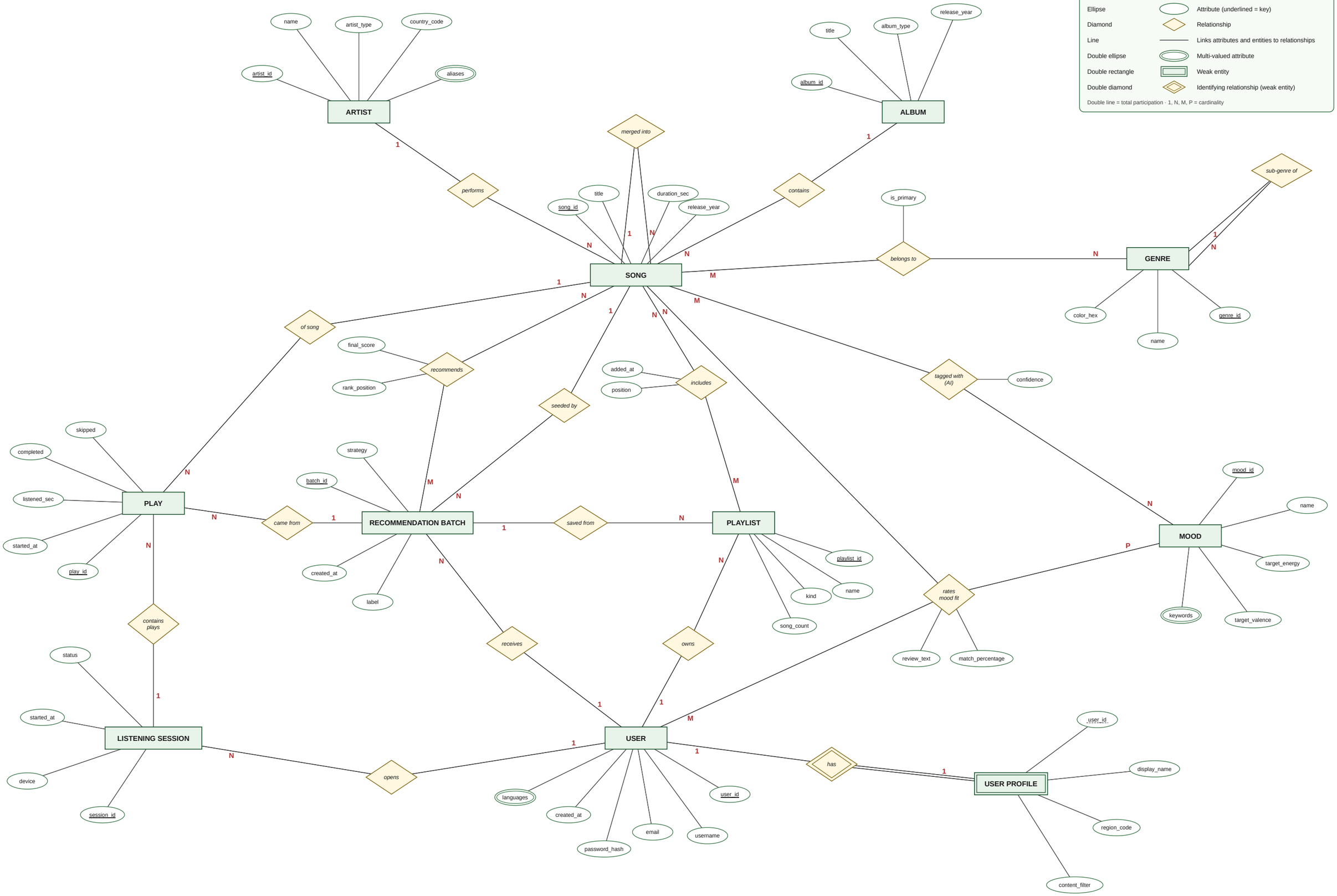
The next page draws the same core schema in **Chen notation**: rectangles are entities, ellipses attributes (key attributes underlined), diamonds relationships, double ellipses multi-valued attributes and a double rectangle the weak entity. Cardinalities are written on the lines (1, N, M, and P for the third side of the ternary relationship).

In the tables (Section 4)	In Chen notation (next page)
Associative tables `song_genres`, `song_moods`, `recommendation_items`, `playlist_songs`	M : N relationships *belongs to*, *tagged with (AI)*, *recommends*, *includes*, with their own attributes (`is_primary`, `confidence`, `rank_position` / `final_score`, `position` / `added_at`)
`song_mood_ratings`	the ternary relationship *rates mood fit* between USER, SONG and MOOD, with `match_percentage` and `review_text`
`user_profiles` (its primary key is also the foreign key to `users`)	weak entity USER PROFILE (double rectangle) with the identifying relationship *has* (double diamond); its key `user_id` comes from the owner (dashed underline) and its participation is total (double line)
`user_languages`, `artist_aliases`, `mood_keywords` (supporting tables, Appendix B)	multi-valued attributes *languages*, *aliases*, *keywords* (double ellipses)
Self-references `songs.merged_into_song_id`, `genres.parent_genre_id`	recursive relationships *merged into* and *sub-genre of* (two lines to the same entity)

The Chen diagram is built from the same schema model and checked automatically: every attribute shown is a real column, and its 18 relationships cover all 24 foreign keys between the core tables. Ellipses show the main attributes; every column is listed in Appendix A.

MUSE — ER Diagram (Chen notation)

11 entities and 18 relationships of the core schema; every relationship is a real foreign key (or a pair/triple of them for M:N and ternary relationships).
Main attributes shown; every column is listed in the Phase 1 report, Appendix A. Cardinality: 1, N, M, P (ternary). Generated and checked against the live MySQL schema.



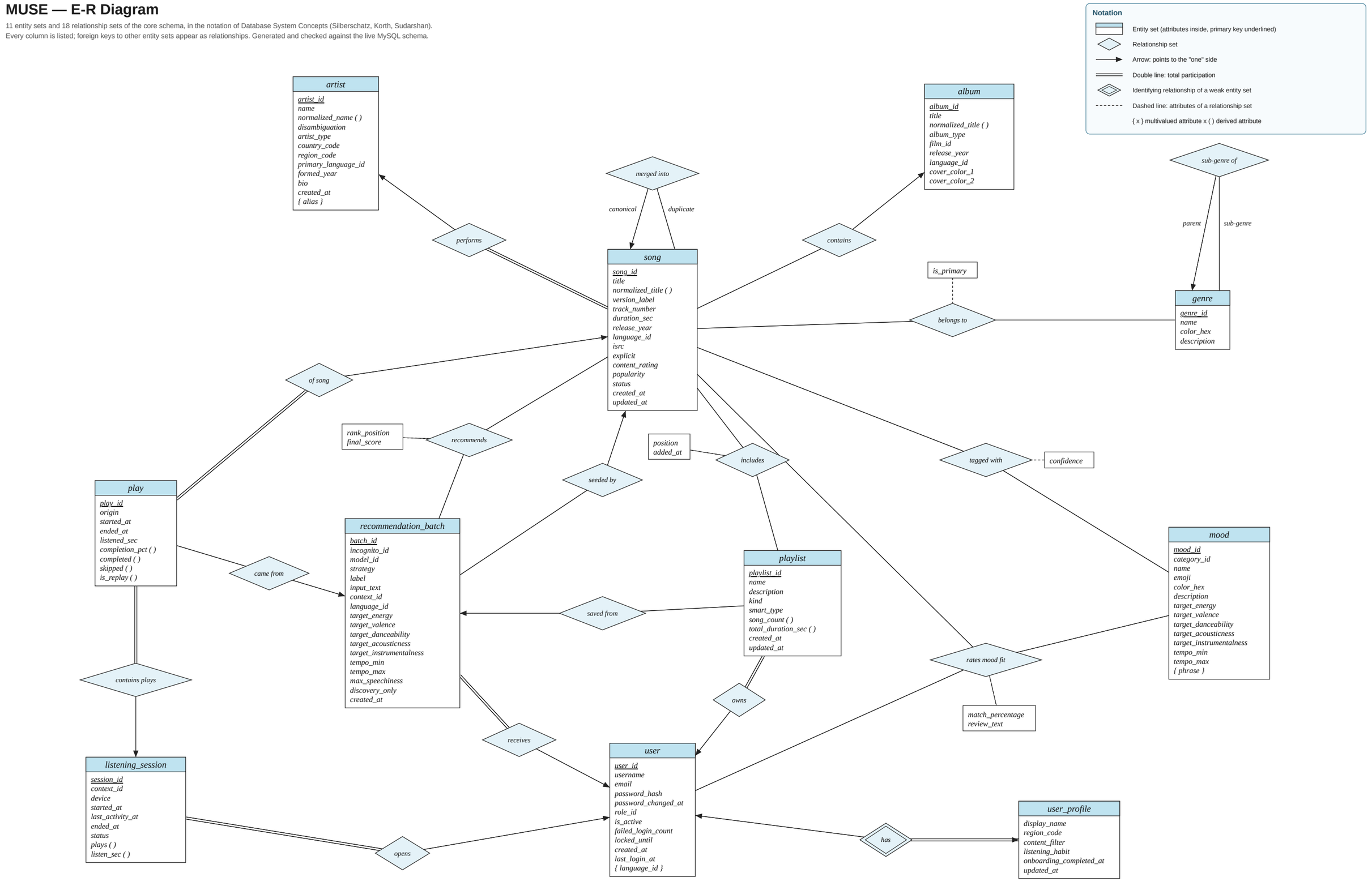
4.2 The same model in textbook (Silberschatz–Korth–Sudarshan) notation

The next page shows the model in the E-R notation of *Database System Concepts*. Each entity set is a box with all its attributes inside (primary key underlined); foreign keys to other entity sets are not listed as attributes because they are the relationship sets. An **arrow** points to the "one" side of a relationship, a **double line** marks total participation (the foreign key is NOT NULL), a **double diamond** is the identifying relationship of the weak entity set `user_profile`, **dashed lines** join relationship attributes (e.g. `match_percentage` of *rates mood fit*), `{x}` is a multivalued attribute and `x()` a derived one. The recursive relationships carry role names (*canonical* / *duplicate*, *parent* / *sub-genre*).

Relationship	Mapping cardinality and participation
performs, *of song*, *contains plays*, *opens*, *receives*, *owns*	many-to-one, total participation of the "many" side (double line)
contains, *came from*, *seeded by*, *saved from*, *merged into*, *sub-genre of*	many-to-one, partial participation (the foreign key may be NULL)
has	one-to-one identifying relationship; every `user_profile` belongs to exactly one `user`
belongs to, *tagged with*, *recommends*, *includes*, *rates mood fit* (ternary)	many-to-many, partial participation, with relationship attributes

MUSE — E-R Diagram

11 entity sets and 18 relationship sets of the core schema, in the notation of Database System Concepts (Silberschatz, Korth, Sudarshan).
Every column is listed; foreign keys to other entity sets appear as relationships. Generated and checked against the live MySQL schema.



5. Normalisation

Normalisation is shown by starting from one un-normalised "listening sheet", the way a naive system might store plays, and decomposing it step by step. The sample rows are real rows from the project's demo data (user *aarav*).

5.1 Un-normalised form (UNF)

user	email	region	languages	song	artist	artist country	album	genres	moods (confidence)	duration	played at	listened	rating
aarav	aarav@muse.dev	IN-TG Telangana	te, hi	Raatbhar	DJ Tara	IN	Neon Mela	EDM, Electronic	Energetic 0.915, Motivational 0.815, Euphoric 0.777, Confident 0.731	232	10-10 18:41	232	4
aarav	aarav@muse.dev	IN-TG Telangana	te, hi	Raatbhar	DJ Tara	IN	Neon Mela	EDM, Electronic	Energetic 0.915, Motivational 0.815, Euphoric 0.777, Confident 0.731	232	10-10 18:44	232	4
aarav	aarav@muse.dev	IN-TG Telangana	te, hi	Ganpati Dhol	Sahyadri Strings	IN	Sahyadri Sunrise	Folk	Euphoric 0.863, Happy 0.854, Motivational 0.801, Energetic 0.686	230	10-11 17:02	230	5

Problems. (1) *Repeating groups / non-atomic values:* `languages`, `genres` and `moods` hold lists. (2) *Update anomaly:* the song facts of "Raatbhar" are repeated on every play; correcting its album means changing many rows. (3) *Insertion anomaly:* a new song cannot be stored until someone plays it. (4) *Deletion anomaly:* deleting the only play of "Ganpati Dhol" in this sheet also deletes everything known about the song and its artist.

5.2 Functional dependencies

The main functional dependencies (FDs) in this data are:

- `user → email, region, display_name, …`
- `region → region_name, country`
- `song → title, duration, release_year, isrc, album, artist, …`
- `artist → artist_name, artist_type, country_code, …`
- `album → album_title, album_type, film, release_year, …`
- `(song, mood, model) → confidence`; `(song, genre) → is_primary`
- `(user, song) → rating` (a user rates a song once; the rating does not depend on a particular play)
- `(user, song, played_at) → listened, session, device, …` (facts of one play)

5.3 First normal form (1NF)

Rule: every attribute holds a single atomic value and there are no repeating groups. Each multi-valued attribute is moved to its own relation, keyed by the owner plus the value:

- PLAY_1NF(user, song, played_at, email, region, region_name, title, artist, artist_country, album, duration, listened, rating)
- USER_LANGUAGE(user, language, priority)
- SONG_GENRE(song, genre, is_primary)
- SONG_MOOD(song, mood, model, confidence) (model = the analysis model version that produced the confidence)

Every cell now holds one value. The relation is in 1NF but still has the update, insertion and deletion anomalies, because user and song facts are repeated on every play.

5.4 Second normal form (2NF)

Rule: 1NF, and no non-key attribute depends on only *part* of a composite candidate key (no partial dependency). The key of `PLAY_1NF` is `(user, song, played_at)`. The partial dependencies are:

Partial dependency	Decomposition
user → email, region, region_name	USER(user, email, region, region_name)
song → title, artist, artist_country, album, duration	SONG(song, title, artist, artist_country, album, duration)
(user, song) → rating	RATING(user, song, rating)
remaining: (user, song, played_at) → listened	PLAY(user, song, played_at, listened)

In the three associative relations of 1NF the non-key attributes depend on the *whole* key (is_primary on (song, genre), confidence on (song, mood, model), priority on (user, language)), so they are already in 2NF. All four anomalies of 5.1 are now removed: a song can exist without plays, and song facts are stored once.

5.5 Third normal form (3NF)

Rule: 2NF, and no non-key attribute depends on another non-key attribute (no transitive dependency key → X → Y). The transitive dependencies left after 2NF and their decomposition:

Transitive dependency	Decomposition (final tables)
user → region → region_name, country	regions(region_code, name, country_code, …); the profile keeps only region_code
song → artist → artist_name, artist_country	artists(artist_id, name, country_code, …); songs keep primary_artist_id
song → album → album_title, album_type, film	albums(album_id, …) and films(film_id, …); songs keep album_id
play → session → device, context, started_at	listening_sessions(session_id, user_id, device, context_id, …); each play keeps session_id
mood → category → category_name	mood_categories(category_id, …); moods keep category_id
user → (login data) / (profile data)	split 1:1 into users (credentials) and user_profiles (display data), both keyed by user_id

Surrogate keys (user_id, song_id, play_id, …) replace the natural keys of the derivation. The natural identities stay enforced as unique keys, for example users.username, users.email, songs.isrc and songs(primary_artist_id, normalized_title, version_label).

5.6 Check of the final core tables

Table	Key	2NF	3NF	Reason
users	user_id	yes	yes	All attributes describe the account; username and email are candidate keys.
user_profiles	user_id	yes	yes	Key is user_id (1:1 with users); region name is in regions.
artists	artist_id	yes	yes*	Region and language details are in their own tables. See note on country_code (5.7).
albums	album_id	yes	yes	Film and language details are in films / languages.
songs	song_id	yes	yes	Artist and album details are in their own tables; only their keys are stored.
genres	genre_id	yes	yes	Single-attribute key; parent genre is a key reference.
song_genres	song_id, genre_id	yes	yes	is_primary depends on the whole key (song, genre).
moods	mood_id	yes	yes	Category details are in mood_categories.
song_moods	song_id, mood_id, model_id	yes*	yes*	confidence depends on the whole key. See note on feature_id (5.7).
song_mood_ratings	user_id, song_id, mood_id	yes	yes	match_percentage and review_text depend on the whole key (user, song, mood); see 5.8.
listening_sessions	session_id	yes	yes*	Context details are in contexts; plays / listen_sec are controlled aggregates.
listening_history	play_id	yes	yes*	Song and session facts are not repeated; user_id and outcome flags are controlled (5.7).
recommendation_batches	batch_id	yes	yes	Targets describe the request itself (copied at request time, not a dependency on a mood).
recommendation_items	batch_id, song_id	yes	yes	Every score depends on the whole key (batch, song).
playlists	playlist_id	yes	yes*	song_count / total_duration_sec are trigger-maintained aggregates (5.7).
playlist_songs	playlist_id, song_id	yes	yes	position and added_at depend on the whole key (playlist, song).

* with the controlled exception explained in 5.7

5.7 Deliberate exceptions (controlled redundancy)

A few attributes are kept although they can be derived, because they are read very often. Each is either computed by MySQL itself or kept consistent by a constraint, trigger or procedure, so it cannot silently disagree with its source:

Attribute(s)	Dependency	How consistency is guaranteed
listening_history.user_id	play → session_id → user_id (transitive)	Composite foreign key (session_id, user_id) → listening_sessions(session_id, user_id): a mismatching user is rejected by the database.

listening_history.completion_pct, completed, skipped, is_replay	derived from listened_sec and the song's duration	Set by triggers trg_lh_bi / trg_lh_bu; values sent by the application are overwritten.
playlists.song_count, total_duration_sec; listening_sessions.plays, listen_sec	aggregates of child rows	Playlists: triggers trg_ps_ai / trg_ps_ad on playlist_songs. Sessions: incremented in the same transaction as each play and recomputed exactly by sp_close_listening_session.
songs.normalized_title, artists.normalized_name, albums.normalized_title, song_genres.primary_song_key	functions of other columns of the same row	Generated columns: MySQL computes them, so they can never drift.

> **Two known deviations, stated honestly.**
> (1) artists.country_code: when an artist has a region_code, the country is also implied by regions.country_code (artist → region_code → country_code). The column is needed for artists without a region (6 of the 31 artists in the seeded demo data, 07_seed.sql), but no constraint currently checks that it matches the region. A strict 3NF fix is to add a unique key regions(region_code, country_code) and a composite foreign key artists(region_code, country_code) to it. In the seeded demo data every artist that has a region carries the matching country.
> (2) song_moods.feature_id records which audio-analysis row a mood was computed from. Within one analysis run it is the same for every mood of a song, i.e. (song_id, model_id) → feature_id, a dependency on part of the key. It is written only by the procedure sp_analyze_song_moods, which inserts all rows of a run from one feature row, so the values stay consistent; a strict 2NF design would move it to a relation keyed by (song_id, model_id).
> Therefore the accurate statement is: **the schema is in 3NF except for these documented, controlled cases.**

5.8 New relation: mood-wise percentage ratings

Requirement. Users rate how well a song fits each mood on a 0–100 scale (for example Happy 92.50 %, Sad 18.00 %), may change their rating later, and must not rate the same song and mood twice. Averages and rating counts per song and mood are needed for mood-based recommendations. These ratings are user opinion, which is a different fact from the AI's prediction in song_moods.confidence.

Un-normalised idea. Storing the ratings as columns of songs (happy_pct, sad_pct, …) or as a list in one cell breaks 1NF, cannot hold one value *per user*, and needs a schema change for every new mood. Adding the user's ratings to song_moods would mix two facts with different keys: (song, mood, model) → AI confidence and (user, song, mood) → user rating.

Functional dependencies. (user_id, song_id, mood_id) → match_percentage, review_text, created_at, updated_at. No attribute depends on a proper subset of the key: the same user gives different values to different moods of one song, different users give different values to the same song and mood, and one user rates the same mood differently for different songs. No non-key attribute determines another.

Result. SONG_MOOD_RATINGS(user_id, song_id, mood_id, match_percentage, review_text, created_at, updated_at) is in 1NF (atomic values), 2NF (no partial dependency) and 3NF (no transitive dependency); its only determinant is the key, so it is also in BCNF. The average and count per song and mood are **not stored**; they are computed by the view v_song_mood_rating_stats (GROUP BY song_id, mood_id), so they can never disagree with the ratings.

Rule	How the database enforces it
One rating per user, song and mood	Composite primary key (user_id, song_id, mood_id); a second insert fails (error 1062).
0 ≤ rating ≤ 100, two decimals	DECIMAL(5,2) and CHECK (match_percentage BETWEEN 0 AND 100).
Valid user, song and mood	Foreign keys to users, songs, moods with the same column types (INT UNSIGNED, INT UNSIGNED, SMALLINT UNSIGNED); ON DELETE CASCADE.
AI prediction is never overwritten	Stored in a different table; no trigger or procedure writes user ratings into song_moods.

6. Final normalised schema

Relational notation for the 16 core relations, with **every** attribute: underlined = primary key, *italic* → table = foreign key, [d] = derived attribute (Section 5.7). Unique keys are listed in Section 2 and Appendix A.

USERS(user_id, username, email, password_hash, password_changed_at, *role_id*→roles, is_active, failed_login_count, locked_until, created_at, last_login_at)

USER_PROFILES(*user_id*→users, display_name, *region_code*→regions, content_filter, listening_habit, onboarding_completed_at, updated_at)

ARTISTS(artist_id, name, normalized_name[d], disambiguation, artist_type, country_code, *region_code*→regions, *primary_language_id*→languages, formed_year, bio, created_at)

ALBUMS(album_id, title, normalized_title[d], album_type, *film_id*→films, release_year, *language_id*→languages, cover_color_1, cover_color_2)

SONGS(song_id, title, normalized_title[d], version_label, *primary_artist_id*→artists, *album_id*→albums, track_number, duration_sec, release_year, *language_id*→languages, isrc, explicit, content_rating, popularity, status, *merged_into_song_id*→songs, created_at, updated_at)

GENRES(genre_id, name, *parent_genre_id*→genres, color_hex, description)

SONG_GENRES(*song_id*→songs, *genre_id*→genres, is_primary, primary_song_key[d])

MOODS(mood_id, *category_id*→mood_categories, name, emoji, color_hex, description, target_energy, target_valence, target_danceability, target_acousticness, target_instrumentalness, tempo_min, tempo_max)

SONG_MOODS(*song_id*→songs, *mood_id*→moods, *model_id*→intelligence_models, confidence, *feature_id*→audio_features, analyzed_at)

SONG_MOOD_RATINGS(*user_id*→users, *song_id*→songs, *mood_id*→moods, match_percentage, review_text, created_at, updated_at)

LISTENING_SESSIONS(session_id, *user_id*→users, *context_id*→contexts, device, started_at, last_activity_at, ended_at, status, plays[d], listen_sec[d])

LISTENING_HISTORY(play_id, *session_id*→listening_sessions, *user_id*→listening_sessions, *song_id*→songs, *batch_id*→recommendation_batches, origin, started_at, ended_at, listened_sec, completion_pct[d], completed[d], skipped[d], is_replay[d])

RECOMMENDATION_BATCHES(batch_id, *user_id*→users, *incognito_id*→incognito_sessions, *model_id*→intelligence_models, strategy, label, input_text, *context_id*→contexts, *seed_song_id*→songs, *language_id*→languages, target_energy, target_valence, target_danceability, target_acousticness, target_instrumentalness, tempo_min, tempo_max, max_speechiness, discovery_only, created_at)

RECOMMENDATION_ITEMS(*batch_id*→recommendation_batches, *song_id*→songs, rank_position, final_score, mood_score, audio_score, preference_score, recency_score, trend_score, context_score, regional_score, exploration_score, repetition_penalty)

PLAYLISTS(playlist_id, *user_id*→users, name, description, kind, smart_type, *source_batch_id*→recommendation_batches, song_count[d], total_duration_sec[d], created_at, updated_at)

PLAYLIST_SONGS(*playlist_id*→playlists, *song_id*→songs, position, added_at)

Together with the supporting relations of Appendix B (for example RATINGS(user_id, song_id, rating, ...), REGIONS(region_code, ...), FILMS(film_id, ...), MOOD_CATEGORIES(category_id, ...), USER_LANGUAGES(user_id, language_id, priority)) these form the normalised schema implemented in MySQL.

7. Summary

Item	Result
Topic	MUSE — mood-based music recommendation database
Entities in the ER diagram	16 core entities (5 associative), 162 attributes
Relationships	24: 1 one-to-one and 23 one-to-many (2 of them recursive); 6 conceptual many-to-many relationships (one of them ternary) resolved by associative entities
Normal form reached	3NF, with the controlled redundancy and the two documented deviations of Section 5.7
Complete implementation	64 tables, 124 foreign keys (Appendix B and C)

Appendix A — Attributes of each entity

Generated from the database catalogue (information_schema). UK1, UK2… mark the columns of each unique key.

users — User, 11 attributes

Key	Attribute	Type	Null	Notes
PK	user_id	int	no	auto-increment
UK2	username	varchar(30)	no	
UK1	email	varchar(120)	no	
	password_hash	char(60)	no	
	password_changed_at	datetime	yes	
FK	role_id	tinyint	no	→ roles
	is_active	tinyint(1)	no	
	failed_login_count	tinyint	no	
	locked_until	datetime	yes	
	created_at	datetime	no	
	last_login_at	datetime	yes	

user_profiles — User profile, 7 attributes

Key	Attribute	Type	Null	Notes
PK FK	user_id	int	no	→ users
	display_name	varchar(60)	no	
FK	region_code	varchar(6)	yes	→ regions
	content_filter	enum(OFF, NO_EXPLICIT, FAMILY)	no	
	listening_habit	enum(BACKGROUND, ACTIVE, MIXED)	yes	
	onboarding_completed_at	datetime	yes	
	updated_at	datetime	no	

artists — Artist, 11 attributes

Key	Attribute	Type	Null	Notes
PK	artist_id	int	no	auto-increment
	name	varchar(100)	no	
UK1	normalized_name	varchar(100)	yes	derived: generated column (from name)
UK1	disambiguation	varchar(80)	no	
	artist_type	enum(SOLO, GROUP, COMPOSER, …)	no	
	country_code	char(2)	no	
FK	region_code	varchar(6)	yes	→ regions
FK	primary_language_id	smallint	yes	→ languages
	formed_year	smallint	yes	
	bio	varchar(400)	yes	
	created_at	datetime	no	

albums — Album, 9 attributes

Key	Attribute	Type	Null	Notes
PK	album_id	int	no	auto-increment
	title	varchar(150)	no	
	normalized_title	varchar(150)	yes	derived: generated column (from title)
	album_type	enum(ALBUM, EP, SINGLE, …)	no	
FK	film_id	int	yes	→ films
	release_year	smallint	no	
FK	language_id	smallint	yes	→ languages
	cover_color_1	char(7)	no	
	cover_color_2	char(7)	no	

songs — Song, 18 attributes

Key	Attribute	Type	Null	Notes
PK	song_id	int	no	auto-increment
	title	varchar(150)	no	
UK1	normalized_title	varchar(150)	yes	derived: generated column (from title)
UK1	version_label	varchar(40)	no	
FK UK1	primary_artist_id	int	no	→ artists
FK	album_id	int	yes	→ albums
	track_number	tinyint	yes	
	duration_sec	smallint	no	
	release_year	smallint	no	
FK	language_id	smallint	yes	→ languages
UK2	isrc	char(12)	yes	
	explicit	tinyint(1)	no	
	content_rating	enum(ALL, TEEN, MATURE)	no	
	popularity	tinyint	no	
	status	enum(ACTIVE, PENDING_ANALYSIS, HIDDEN, …)	no	
FK	merged_into_song_id	int	yes	→ songs
	created_at	datetime	no	
	updated_at	datetime	no	

genres — Genre, 5 attributes

Key	Attribute	Type	Null	Notes
PK	genre_id	smallint	no	auto-increment
UK1	name	varchar(40)	no	
FK	parent_genre_id	smallint	yes	→ genres
	color_hex	char(7)	no	
	description	varchar(200)	yes	

song_genres — Song–Genre (associative), 4 attributes

Key	Attribute	Type	Null	Notes
PK FK	song_id	int	no	→ songs
PK FK	genre_id	smallint	no	→ genres
	is_primary	tinyint(1)	no	
UK1	primary_song_key	int	yes	derived: generated column (song_id when is_primary)

moods — Mood, 13 attributes

Key	Attribute	Type	Null	Notes
PK	mood_id	smallint	no	auto-increment
FK	category_id	tinyint	no	→ mood_categories
UK1	name	varchar(30)	no	
	emoji	varchar(8)	no	
	color_hex	char(7)	no	
	description	varchar(200)	no	
	target_energy	decimal(4,3)	no	
	target_valence	decimal(4,3)	no	
	target_danceability	decimal(4,3)	no	
	target_acousticness	decimal(4,3)	no	
	target_instrumentalness	decimal(4,3)	no	
	tempo_min	decimal(5,1)	no	
	tempo_max	decimal(5,1)	no	

song_moods — Song–Mood (associative), 6 attributes

Key	Attribute	Type	Null	Notes
PK FK	song_id	int	no	→ songs
PK FK	mood_id	smallint	no	→ moods
PK FK	model_id	smallint	no	→ intelligence_models
	confidence	decimal(4,3)	no	
FK	feature_id	int	no	→ audio_features
	analyzed_at	datetime	no	

song_mood_ratings — Mood-match rating (associative), 7 attributes

Key	Attribute	Type	Null	Notes
PK FK	user_id	int	no	→ users
PK FK	song_id	int	no	→ songs
PK FK	mood_id	smallint	no	→ moods
	match_percentage	decimal(5,2)	no	
	review_text	varchar(280)	yes	
	created_at	datetime	no	
	updated_at	datetime	no	

listening_sessions — Listening session, 10 attributes

Key	Attribute	Type	Null	Notes
PK UK1	session_id	bigint	no	auto-increment
FK UK1	user_id	int	no	→ users
FK	context_id	tinyint	yes	→ contexts
	device	varchar(30)	no	
	started_at	datetime	no	
	last_activity_at	datetime	no	
	ended_at	datetime	yes	
	status	enum(OPEN, CLOSED)	no	
	plays	smallint	no	derived: aggregate of the session's plays
	listen_sec	int	no	derived: aggregate of the session's plays

listening_history — Play (listening history), 13 attributes

Key	Attribute	Type	Null	Notes
PK	play_id	bigint	no	auto-increment
FK	session_id	bigint	no	→ listening_sessions
FK	user_id	int	no	→ listening_sessions
FK	song_id	int	no	→ songs
FK	batch_id	bigint	yes	→ recommendation_batches
	origin	enum(HOME, MOOD, CONTEXT, …)	no	
	started_at	datetime	no	
	ended_at	datetime	yes	
	listened_sec	smallint	no	
	completion_pct	decimal(5,2)	no	derived: set by trigger from listened_sec and song duration
	completed	tinyint(1)	no	derived: set by trigger
	skipped	tinyint(1)	no	derived: set by trigger
	is_replay	tinyint(1)	no	derived: set by trigger

recommendation_batches — Recommendation batch, 20 attributes

Key	Attribute	Type	Null	Notes
PK	batch_id	bigint	no	auto-increment
FK	user_id	int	no	→ users
FK	incognito_id	bigint	yes	→ incognito_sessions
FK	model_id	smallint	no	→ intelligence_models
	strategy	enum(MOOD, TEXT, CONTEXT, …)	no	
	label	varchar(120)	yes	
	input_text	varchar(280)	yes	
FK	context_id	tinyint	yes	→ contexts
FK	seed_song_id	int	yes	→ songs
FK	language_id	smallint	yes	→ languages
	target_energy	decimal(4,3)	no	
	target_valence	decimal(4,3)	no	
	target_danceability	decimal(4,3)	no	
	target_acousticness	decimal(4,3)	no	
	target_instrumentalness	decimal(4,3)	no	
	tempo_min	decimal(5,1)	no	
	tempo_max	decimal(5,1)	no	
	max_speechiness	decimal(4,3)	no	
	discovery_only	tinyint(1)	no	
	created_at	datetime	no	

recommendation_items — Recommended song (associative), 13 attributes

Key	Attribute	Type	Null	Notes
PK FK UK1	batch_id	bigint	no	→ recommendation_batches
PK FK	song_id	int	no	→ songs
UK1	rank_position	smallint	no	
	final_score	decimal(6,4)	no	
	mood_score	decimal(6,4)	yes	
	audio_score	decimal(6,4)	no	
	preference_score	decimal(6,4)	no	
	recency_score	decimal(6,4)	yes	
	trend_score	decimal(6,4)	no	
	context_score	decimal(6,4)	yes	
	regional_score	decimal(6,4)	yes	
	exploration_score	decimal(6,4)	no	
	repetition_penalty	decimal(6,4)	no	

playlists — Playlist, 11 attributes

Key	Attribute	Type	Null	Notes
PK	playlist_id	int	no	auto-increment
FK UK1	user_id	int	no	→ users
UK1	name	varchar(80)	no	
	description	varchar(255)	yes	
	kind	enum(MANUAL, SMART, SAVED_MIX, …)	no	
	smart_type	varchar(30)	yes	
FK	source_batch_id	bigint	yes	→ recommendation_batches
	song_count	smallint	no	derived: maintained by triggers on playlist_songs
	total_duration_sec	int	no	derived: maintained by triggers on playlist_songs
	created_at	datetime	no	
	updated_at	datetime	no	

playlist_songs — Playlist–Song (associative), 4 attributes

Key	Attribute	Type	Null	Notes
PK FK	playlist_id	int	no	→ playlists
PK FK	song_id	int	no	→ songs
	position	smallint	no	
	added_at	datetime	no	

Appendix B — The remaining 48 tables of the database

The database has 64 tables. The table below assigns each one that is not drawn in the 16-entity diagram to the core entity it supports. `schema_migrations` only records which schema versions were applied.

Table	Supports	Primary key	References
roles	users	role_id	—
permissions	users	permission_code	—
role_permissions	users	role_id, permission_code	permissions, roles
auth_sessions	users	session_id	users
user_languages	users	user_id, language_id	languages, users
audit_logs	users	audit_id	users
schema_migrations	users	version	—
regions	user_profiles	region_code	languages
languages	user_profiles	language_id	—
artist_aliases	artists	alias_id	artists
user_artist_preferences	artists	user_id, artist_id	artists, intelligence_models, users
album_artists	albums	album_id, artist_id	albums, artists
films	albums	film_id	languages
song_artists	songs	song_id, artist_id, role	artists, songs
song_media	songs	media_id	music_sources, songs
music_sources	songs	source_id	music_sources
source_records	songs	record_id	albums, artists, films, music_sources, pipeline_jobs, songs
pipeline_jobs	songs	job_id	music_sources, pipeline_jobs, users
pipeline_job_logs	songs	log_id	pipeline_jobs, source_records
review_queue	songs	review_id	pipeline_jobs, songs, source_records, users
trend_snapshots	songs	snapshot_id	pipeline_jobs, regions
user_genre_preferences	genres	user_id, genre_id	genres, intelligence_models, users
mood_categories	moods	category_id	—
mood_keywords	moods	keyword_id	moods
contexts	moods	context_id	—
context_keywords	moods	phrase	contexts
context_moods	moods	context_id, mood_id	contexts, moods
user_mood_preferences	moods	user_id, mood_id	intelligence_models, moods, users
user_context_preferences	moods	user_id, context_id	contexts, users
audio_features	song_moods	feature_id	intelligence_models, songs
mood_segments	song_moods	segment_id	intelligence_models, moods, songs
song_mood_curations	song_moods	song_id, mood_id	moods, songs, users
intelligence_models	song_moods	model_id	—
model_parameters	song_moods	model_id, param_code	intelligence_models
incognito_sessions	listening_sessions	incognito_id	moods, users
incognito_events	listening_sessions	event_id	incognito_sessions, songs
search_history	listening_sessions	search_id	users
playback_events	listening_history	event_id	listening_history
ratings	listening_history	user_id, song_id	songs, users
user_likes	listening_history	user_id, song_id	songs, users
song_dismissals	listening_history	user_id, song_id	recommendation_batches, songs, users
recommendation_batch_moods	recommendation_batches	batch_id, mood_id	moods, recommendation_batches
weekly_worlds	recommendation_batches	world_id	languages, moods, pipeline_jobs, recommendation_batches, regions, users, world_archetypes
world_archetypes	recommendation_batches	archetype_code	—
archetype_moods	recommendation_batches	archetype_code, mood_id	moods, world_archetypes

world_feedback	recommendation_batches	user_id, world_id	users, weekly_worlds
recommendation_reasons	recommendation_items	batch_id, song_id, reason_code	reason_types, recommendation_items
reason_types	recommendation_items	reason_code	—

Appendix C — Complete ER diagram of all tables

The next page shows every table of the implementation with all 124 foreign keys, grouped in eight functional areas, in the same notation as Section 4. It is printed on an A1 page and is fully vector, so it can be zoomed. The same diagram, plus one page per functional area at full size, is provided separately as `MUSE_ERD_Full_Schema.pdf`.

MUSE — Complete Database ERD

All 64 tables and all 124 foreign-key relationships. Key columns only (PK, FK, unique keys); every other attribute is in docs/DATA_DICTIONARY.md. Generated from the live MySQL schema (baseline 01–06 + migrations 0001, 0002, 0003, 0004) by backend/scripts/erd/build_erd.py.

